

STM32F1 Emulator

Technical Reference Manual

Development Documentation

Full-system STM32F1 emulation in one WASM module

`stm32f1-emu@3.2.0` • ESM • Node ≥ 18

Source code: <https://github.com/danish9661/STM32F1-emulator>

Live demo: <https://danish9661.github.io/STM32F1-emulator/>

September 2026

Metric	Value
Package	<code>stm32f1-emu@3.2.0</code> , ESM, Node ≥ 18
CPU	Native Rust ARM Cortex-M3 Thumb-2 + FlatMemory
Peripherals	30+ Rust models, builtin F103 map + SVD maps
Test suite	<code>tests/test_all.mjs</code> 772 asserts , canary 39/39
Demo firmware	~30 Arduino sketches + 2 bare-metal OTG programs
Performance	~ 70M IPS headless (200M in ~2.8 s, MPU on)

Audience: firmware developers verifying code without hardware; web developers embedding an MCU in a page; students learning Cortex-M; integrators wiring the emulator into other tools.

Companion: `docs/STM32F1_Guide.md` (same 15-part structure, website Guide entry, Markdown). This `.tex` is the print source; build with `pdflatex` (twice for the TOC).

Contents

1	Part 1: Architecture Overview and Design Philosophy	2
1.1	Project summary	2
1.2	Design goals	2
1.3	System characteristics	2
1.4	Execution modes	3
1.5	Boot sequences	3
2	Part 2: Building and Running	3
2.1	Prerequisites	3
2.2	Install and build	3
2.3	Running tests	4
2.4	Running firmware	4
2.5	GDB debugging	4
2.6	WebSocket bridge	4
2.7	Custom JS peripherals	5

3	Part 3: Command-Line Reference	5
3.1	Usage syntax	5
3.2	Options	5
3.3	Config file	5
3.4	GDB server options	6
3.5	WebSocket server options	6
4	Part 4: Memory Maps	6
4.1	Flash / SRAM per chip	6
4.2	System regions	6
4.3	Peripheral base addresses	7
5	Part 5: CPU Core	8
5.1	5.1 Instruction set	8
5.2	5.2 Exception handling	8
5.3	5.3 Timing model	9
5.4	5.4 CPU state reference	9
6	Part 6: Peripherals and Virtual Devices	9
6.1	6.1 GPIO	9
6.2	6.2 USART	10
6.3	6.3 SPI	11
6.4	6.4 I2C	11
6.5	6.5 TIM	12
6.6	6.6 ADC	12
6.7	6.7 DAC	13
6.8	6.8 DMA	13
6.9	6.9 CAN	14
6.10	6.10 RTC / BKP / PWR / FLASH	14
6.11	6.11 FSMC / SDIO	15
6.12	6.12 USB FS / OTG_FS	15
6.13	6.13 NVIC / SysTick / SCB / MPU / Debug	16
6.14	6.14 IRQ number table	17
7	Part 7: Tested Firmware and Test Suite	18
7.1	7.1 Test suite	18
7.2	7.2 RP2040/RP2350 comparison	18
7.3	7.3 Demo firmwares	18
7.4	7.4 Known divergence	19
8	Part 8: JS API Reference	19
8.1	8.1 Package layout	19
8.2	8.2 High-level API	19
8.3	8.3 Low-level API	20
8.4	8.4 DMA API	20
8.5	8.5 ADC + TIM API	20
8.6	8.6 Servers and debug	20
9	Part 9: Debugging	21
9.1	9.1 GDB remote debugging	21
9.2	9.2 Debug output and watch tools	21

10 Part 10: Storage, Networking, Multi-Device	22
10.1 10.1 Flash images	22
10.2 10.2 SD card	22
10.3 10.3 Networking	22
10.4 10.4 Multi-device	22
11 Part 11: Performance	23
12 Part 12: Design Decisions and Trade-Offs	23
13 Part 13: Repository Structure	24
14 Part 14: Version History	25
14.1 Version history	25
15 Part 15: External References	25
15.1 External references (annotated)	25
15.2 Glossary	26
15.3 Document history	26

1 Part 1: Architecture Overview and Design Philosophy

1.1 Project summary

`stm32f1-emu` executes real firmware — ELF, Intel HEX, or raw binaries produced by Arduino/STM32duino, STM32Cube, libopencm3, xpack-gcc, and other toolchains — **without modification**:

1. **Native Rust CPU** (`src/cpu/`): ARM Cortex-M3 Thumb-2 interpreter + guest RAM (`FlatMemory`). Zero JS crossings per instruction.
2. **Rust peripherals**: GPIO, USART, SPI, I2C, TIM, ADC, DAC, DMA, CAN, RTC, CRC, NVIC, EXTI, FLASH, PWR, BKP, IWDG, WWDG, FSMC, SDIO, USB FS, USB OTG_FS (F105), DWT/ITM, SWD/JTAG debug slice. Two maps agree on real F103 addresses: a builtin hardcoded table plus `from_svd()` (ships `svd/STM32F103.svd` + `svd/STM32F105xx.svd`; SVDs missing core blocks get NVIC/SysTick/SCB auto-registered at fixed `0xE000Exxx`).
3. **JS orchestration** (`pkg/emulator.js`, `pkg/cli.mjs`, `site/worker.js`, `site/index.html`): per batch (`DEFAULT_MAX_BATCH 20000`): `rustcpu_dma_pump()` → `rustcpu_run(batch)` (exact count back) → `step_batch(count)` (peripheral tick, once per batch, instruction-delta) → `rustcpu_dma_pump()` → `rustcpu_dispatch()` (≤ 64 IRQs via `intr_next`).

1.2 Design goals

1. **Boot real firmware unmodified.** Any ELF/HEX/BIN for the F1 family should load and execute without patching (one documented `hi2c->Mode` RAM patch remains, see §??).
2. **Register-level peripheral fidelity.** Every modeled peripheral reproduces the documented RM0008 register layout; firmware that polls status bits, masks interrupts, or programs DMA gets correct values.
3. **One WASM module, zero hot-path crossings.** CPU + peripherals live in Rust; JS drives batches only. DMA pumps and IRQ dispatch run fully in Rust.
4. **Interactive debugging.** GDB RSP stub, SWD/JTAG slice, symbol resolution, fault snapshots.
5. **Board coverage without SVD sprawl.** Register-identical clones (GD32) are a chip table (sizes + IDCODE), not new maps; only genuinely different silicon (F105: CAN2 + OTG_FS) needs SVD.
6. **Embeddable API.** `rp2040js`/Wokwi-style wrapper (`STM32F1`), raw bridge (`createEmulator`), WebSocket bridge, custom JS peripherals.

1.3 System characteristics

Feature	Details
Language	Rust (wasm-bindgen) + JS (ESM, no bundler needed)
Build	<code>wasm-pack build --target web</code> (pinned <code>binaryen 132</code>)
Hosts	Node ≥ 18 , any modern browser (Worker + rAF paths)
CPU	Cortex-M3 Thumb-2: IT, ThumbExpandImm, bitfield, UDIV/SDIV, LDREX/STREX, TBB/TBH, MPU, MSP/PSP banks
Memory	64K–256K flash + 20K–64K SRAM per chip; FSMC; bitband; MPU
Peripherals	30+ modules (see §4.3)
Firmware	ELF32 ARM, Intel HEX, raw binary @ <code>0x08000000</code>

Feature	Details
Debugging	GDB RSP, SWD/JTAG slice, <code>takeFault()</code> , map symbols
Clock model	Instruction-budget: 1 instr = 1 cycle
Performance	~70M IPS headless; browser loop frame-budgeted (80 ms)
Storage	SPI flash, I2C EEPROM, FSMC NOR, SDHC images
Virtual devices	Transaction events + host injection
Output model	Firmware UART on <code>getUartOutput()</code> ; diagnostics on console

1.4 Execution modes

1. **Headless CLI** (`pkg/cli.mjs`): firmware + YAML config, run to an instruction cap, print UART + SUMMARY. The CI/canary path.
2. **Library loop** (`createEmulator().run()/step()`): adaptive 20K (IRQ/DMA pending, ~1.1 ms latency) / 50K idle batches, or fixed `batch_size`.
3. **Browser worker** (`site/worker.js`): emulation off the main thread at ~60 fps, frame-budgeted stepping.
4. **GDB-driven** (`pkg/gdbstub.mjs`): the core steps under debugger control (chunked batches between RSP polls).

1.5 Boot sequences

```

parse ELF (segments+symbols) / HEX (base+data) / BIN (@0x08000000)
reset_ext_devices() -> register_ext_devices (flash/EEPROM/OLED/LCD/...)
init() [builtin F103] or init_svd(xml) [F105/any F1 SVD]
set_dbg_idcode(chip) -> rustcpu_init(SP, PC, flash, ram) ->
    rustcpu_load(regions)
run/step loop: DMA pump -> run batch -> step_batch tick -> DMA pump ->
    dispatch IRQs

```

Vector SP/PC are validated ($SP \neq 0$, PC Thumb bit set); ELF `.data` LMA→VMA copies are honored. `reset() = board_nrst()` (model) + `bootCpu()` (CPU/RAM reload); `board_nrst()` is deliberately a *half* reset (never rebuilds the map — an F105 board must not silently become F103). BOOT0 high at reset claims the AN3155 USART1 bootloader path instead of main flash.

2 Part 2: Building and Running

2.1 Prerequisites

Runtime: Node.js ≥ 18 (ESM); any modern browser. **Build (only to rebuild the WASM):**

`rustc 1.97.1`, `wasm-pack 0.14.0`, `binaryen version_132` at `~/.local/binaryen/binaryen-version_132/bin` (persistent — never `/tmp`, the box wipes it and `wasm-pack` silently falls back to a stale `wasm-opt`).

Optional: `arduino-cli` + STM32duino 2.12.0 (firmware rebuilds), `xpack-gcc 14.2.1` (bare-metal OTG), `arm-none-eabi-gdb 15` (live GDB), Playwright Chromium (browser tests).

2.2 Install and build

```

npm install stm32f1-emu
# Rebuild Rust -> WASM (exact flags matter for the CI byte-exact guard)
PATH=~/.local/bin:$(npm config get bin_prefix)/bin:$PATH \
RUSTFLAGS="--remap-path-prefix=$HOME=/build" \
wasm-pack build --target web --out-dir pkg
cp pkg/stm32_bluepill_wasm_bg.wasm pkg/stm32_bluepill_wasm.js site/
cp pkg/emulator.js site/emulator.js      # + stm32f1.js / .d.ts when changed
node site/sync-docs.mjs                  # after any docs/*.md edit

```

2.3 Running tests

```

node tests/test_all.mjs      # 772 unit asserts (peripherals in isolation)
node tests/canary.mjs        # 39/39 firmware checks (~25 s at 100M)
echo -n "AB" | node pkg/cli.mjs \
  --config=tests/arduino_periph_test/config.yaml --max=200000000 # 200M
  ~3 s
node tests/test_emulator_js.mjs # browser-loop path
cargo test --release --lib cpu::census && python3 tests/census_16.py \
  && python3 tests/census_32.py
python3 tests/fuzz_diff.py --cases 200 --seed 1 # diff fuzz vs Unicorn

```

2.4 Running firmware

```

# Headless: ELF/HEX/BIN, instruction cap, UART on stdout
node pkg/cli.mjs firmware.elf --max=200000000
echo -n "AB" | node pkg/cli.mjs \
  --config=tests/arduino_periph_test/config.yaml --max=200000000
# Browser demo
python3 -m http.server -d site 8765 # open /index.html, pick a preset,
  Run

```

```

// Library (Node or browser)
import { STM32F1 } from 'stm32f1-emu';
const mcu = await STM32F1.fromELF(firmwareBytes);
mcu.usart1.onData = (b) => process.stdout.write(String.fromCharCode(b));
await mcu.execute(5_000_000);

```

2.5 GDB debugging

```

import { serveGdb } from 'stm32f1-emu/gdb';
const srv = await serveGdb({ firmware, port: 1234, chip: 'gd32f103c8' });

```

```

arm-none-eabi-gdb firmware.elf -ex 'target remote :3333'
# break loop / continue / info registers / x/8xw $sp / stepi
# watch *(char*)0x20000100

```

2.6 WebSocket bridge

```

npm install ws
node pkg/ws-server.mjs site/arduino_periph_test.elf --port=8080
# open http://localhost:8080/ws-viewer.html

```

2.7 Custom JS peripherals

```
// Browser / library: shadow any window (last registration wins)
emu.addJsPeripheral(0x40007C00, 0x100,
  (addr, size) => addr === 0x40007C00 ? 0xC0FFEE : 0,
  (addr, value, size) => { /* ... */ });
```

```
# CLI plugin module (default export [{base,size,read,write}]):
node pkg/cli.mjs firmware.elf --periph-plugin=./my_periph.mjs
```

3 Part 3: Command-Line Reference

3.1 Usage syntax

```
stm32f1-emu <firmware> [max_instructions] [options]
stm32f1-emu --config=<path.yaml> [max_instructions] [options]
```

Firmware formats: `.elf` (segments + symbols auto-resolved), `.hex` (Intel HEX), `.bin` (raw @ 0x08000000).

3.2 Options

Flag	Default	Description
<code>-config</code>	—	YAML config (repeatable, merged)
<code>-max</code>	1000000	Max instructions (<code>MAX_INST</code>)
<code>-map</code>	—	GNU ld map for PC → symbol
<code>-uart</code>	0x40013800	UART base for stdin RX
<code>-regs</code>	off	Dump registers every batch
<code>-verbose</code>	off	Peripheral traces (very noisy)
<code>-periph-plugin</code>	—	JS peripheral plugin module
<code>-h/-help</code>	—	Usage text

3.3 Config file

```
cpu: { vector_table: 0x08000000, svd: "../..svd/STM32F103.svd" }
regions:
  - { start: 0x08000000, size: 0x20000, load: "build/fw.ino.elf" }
  - { start: 0x20000000, size: 0x5000 }
devices:
  i2c_eeprom: [{ peripheral: "I2C1", addr: "0x50", file:
    "build/eeprom.bin" }]
  i2c_oled:    [{ peripheral: "I2C1", addr: "0x3C", width: 128, height:
    64 }]
  spi_flash:  [{ peripheral: "SPI1", jedec_id: "0xEF4016",
    file: "build/spi_flash.bin", cs: "PA4" }]
  lcd:        [{ peripheral: "SPI1", cs: "PA1" }]
  touchscreen: [{ peripheral: "SPI1", touch_detected_pin: "PA3", cs:
    "PA2" }]
  sd_card:    [{ peripheral: "SDIO", file: "sd.img" }]
```

Devices register **before** `init()`. The stdin loop gates `uart_rx_byte` on `uart_rx_pending==0` and DMA-busy, so the DMA-RX byte A is never eaten by the UART-RX test. Faults use `raise_fault(3) + PC+2` when symbols exist; the `canRxArmed` CAN inject is ELF-symbol resolved (never hardcoded).

3.4 GDB server options

`serveGdb({firmware?, chip?, flash_size?, ram_size?, vector_table?, ext_devices?, port?=1234, chunk?=20000}) → {port,emu,close}`. RSP over TCP: ? g G p P m M Z0/z0 Z2/Z3/Z4 c s vCont Hc/Hz, 17-register g (r0–r12, sp, lr, pc, xpsr), halfword-masked BKPT, hex lengths, T05watch:/rwatch:/awatch: stops.

3.5 WebSocket server options

`node pkg/ws-server.mjs <firmware.elf> [-port=8080] [-max=N]:serves site/ over HTTP, streams emulator frames over WS. Inbound JSON: uart_rx {addr?,byte}, gpio_set {port,pin,high}, can_inject, board_reset, board_boot0 {high}. Outbound hello + {e,p,fps,t} frames. Idle with no clients.`

4 Part 4: Memory Maps

4.1 Flash / SRAM per chip

Chip	Flash	RAM	IDCODE	Notes
stm32f103c8	64K	20K	0x10016410	Reference Blue Pill
stm32f103cb / maple_mini	128K	20K	0x10016410	Maple: D33=PB1, BUT=PB8, DFU
nucleo_f103rb	128K	20K	0x10016410	D13=PA5, B1=PC13, Serial=USART2
stm32f103rc	256K	48K	0x10016410	HD: DAC/FSMC/ADC3/UART4-5/SPI3/T
gd32f103c8/cb/rb	64K/128K	20K	0x2BA01477	Clone contract
F105 ({name,svd})	256K	64K	0x10016418	CAN2 + OTG_FS; core auto-registered

Vector table default `0x08000000` (Maple DFU-offset boot uses `0x08005000`). SRAM base `0x20000000`. Flash is guest-executable/read-only (guest stores drop; `load()` bypasses); unmapped reads return 0 + a **bad**-address latch.

4.2 System regions

Range	Use
0x08000000+	Flash (per-chip size)
0x20000000+	SRAM (per-chip size)
0x1FFFF7E8	96-bit UID (constant, writes ignored)
0x40000000–0xB0000000	Peripherals (in-Rust bus)
0xE0000000–0xE1000000	System (NVIC/SysTick/SCB/DWT/ITM/debug)
0x42000000–0x44000000	Bitband alias to 0x40000000
0x60000000–0xA0001000	FSMC external memory (7 banks)
0x40006000–0x40006400	USB PMA (byte-exact window)
0x50000000–0x50005000	USB OTG_FS (F105; FIFOs at +EP*0x1000)

4.3 Peripheral base addresses

Base	Peripheral	Notes
0x40000000	TIM2	General-purpose 16-bit
0x40000400	TIM3	+ PWM/DMA-burst demo
0x40000800	TIM4	1 ms RTOS tick
0x40000C00	TIM5	HD/CL superset
0x40001000/1400	TIM6/TIM7	Basic timers
0x40002800	RTC	Counter/alarm, runs in STOP
0x40002C00	WWDG	Window watchdog, EWI → IRQ0
0x40003000	IWDG	Independent watchdog, runs in STOP
0x40003800/3C00	SPI2/SPI3	SPI3 = HD superset
0x40004400	USART2	Nucleo/RC native Serial
0x40004800	USART3	
0x40004C00/5000	UART4/UART5	HD/CL superset
0x40005400/5800	I2C1/I2C2	Master + slave/10-bit/PEC/ALERT
0x40005C00	USB FS	Device, PMA to 0x40006400
0x40006400/6800	CAN1/CAN2	Shared banks
0x40006C00	BKP	Backup regs + tamper → IRQ2
0x40007000	PWR	PVD → EXTI16, PDDS/STOP/STANDBY
0x40007400	DAC	CH1→PA4/CH2→PA5 + ADC loopback
0x40010000	AFIO	Remap/EXTI-select/SWJ_CFG
0x40010400	EXTI	20 lines, SWIER
0x40010800–11400	GPIOA–D	+ 0x40011800 GPIOE (HD)
0x40012400/2800	ADC1/ADC2	+ 0x40013C00 ADC3 (HD)
0x40012C00	TIM1	Advanced (BDTR/MOE/break)
0x40013000	SPI1	Flash/LCD/touch/WS2812 host
0x40013800	USART1	Default <code>uart_addr</code>
0x40018000	SDIO	SDHC/MMC host, IRQ49, DMA2 CH4
0x40020000	DMA1	7 channels (streams 0–6)
0x40020400	DMA2	5 channels (streams 7–11)
0x40021000	RCC	Enables/resets + clock query
0x40022000	FLASH	Unlock/program/erase/OBR/WRP
0x40023000	CRC	CRC-32
0x60000000...	FSMC	7 banks; regs at 0xA0000000+
0xE0000000	ITM	Stimulus port 0 → <code>ItmByte</code>
0xE0001000	DWT	CYCCNT (wait-state aware)
0xE000E100...	NVIC	97 IRQs, fairness
0xE000E010...	SysTick	Debt model (irq –1)
0xE000ED00...	SCB	CPUID/VTOR/SHPR/CFSR/HFSR + MPU
0xE000E008	ACTRL	RW store (mask 0x7)
0xE000EF00	STIR	WO: pends any IRQ
0xE0042000	DBGMCU	RO per-chip IDCODE
0xE000EF90...	MPU	8 regions, AP/XN
0xE000EDF0...	Debug	DHCSR/DCRSR/DEMCR + DP/AP/JTAG

5 Part 5: CPU Core

Native Rust Cortex-M3 (`src/cpu/`): `Cpu + Regs` (r0–r15, xPSR, PRIMASK, CONTROL, banked MSP/PSP) + `FlatMemory` + `thumb.rs` decoder/executor (~2.5K lines, every encoding GAS/objdump/Capstone oracle verified; unknown → loud `CpuFault{pc,op1,op2,len}`, never silent).

5.1 5.1 Instruction set

Category	Instructions	Details
Data processing	ADD ADC SUB SBC RSB MOV MVN MUL MLA MLS	ADCS/SBCS set
Comparison	CMP CMN TST TEQ	Flags only; TST:
Logical	AND ORR ORN EOR BIC	S-form writes C
Shift/rotate	LSL LSR ASR ROR RRX	Imm #0 = 32/R
Move wide	MOVW MOVT ADDW SUBW	i-bit; ADDW/S
Bitfield	SBFX UBFX BFI BFC	1sb+w>32 UNPR
Saturate	SSAT USAT (+16 duals)	sat direct vs N-
Divide/multiply	SDIV UDIV SMULL UMULL SMLAL UMLAL	Div0 → 0 (M3 D
Branches	B Bcc BL BLX CBZ	B.W NOT(J^S) J
Table branch	TBB TBH	Value-indexed
Loads/stores	LDR STR LDRB/STRB LDRH/STRH LDRSB/LDRSH + .W wide + reg-shifted	T2-imm12/T3-li
Multiple	LDMIA STMIA LDMDB STMDB (+IB/DA)	Keyed off P+U (
Double	STRD LDRD (incl. post-indexed)	LDRD-into-PC i
Exclusive	LDREX STREX LDREXB/H STREXB/H CLREX	Exact-address re
Stack	PUSH POP	PC → branch (i
System	SVC BKPT UDF MRS MSR CPS DMB/DSB/ISB DBG NOP WFI/WFE	SVC → driver st
Misc	SXTH SXTB UXTH UXTB REV REV16 REVSH ADR RBIT CLZ	M3 has RBIT/C
IT blocks	IT+1-4 payload	it_ok consume;

Dispatch: 256-entry table by `instr[15:8]` (O(1), Bramble-style) + `len(op)` 32-bit prefix check (`op»11 ∈ 11101,11110,11111`) → the matching `exec16/exec32` arm. Every arm shape-checked — reserved and UNPREDICTABLE shapes fault loudly (census + fuzz prove it).

5.2 5.2 Exception handling

Entry (`take_exception(sys,mem,irq)`): push 8-word frame `R0,R1,R2,R3,R12,LR,retPC,xPSR` (T-bit) on the current SP (PSP when `thread+PSP` — RTOS works); switch to MSP; `LR = 0xFFFFFFFF1` (nested handler) / `0xFFFFFFFFD` (thread-PSP) / `0xFFFFFFFF9` (thread-MSP); `ipsr = 16+irq`; IT saved, sleep cleared, `exclusive` cleared; thread bank advanced past the frame (FreeRTOS slide fix); `VC_HARDERR` halt check; `PC = VTOR + vector`.

Return (`exception_return` on `0xFFFFFFFF0...` in BX/POP-PC): rejects non-F1/F9/FD with fault; **unstacks from live r13 for F1/F9 (the stale-bank bug that caused phantom reboots — banks pointed at consumed inner frames after nesting)** and the PSP bank for FD; restores `r0–r3/r12/lr/retpc/xPSR/SP`; re-syncs banks; `CONTROL.SPSEL`-coherent; pops IT/exception stacks; clears the IABR bit; balances the NVIC pop; re-pends exactly one SysTick debt tick; forces `r15 = retpc|1` (FreeRTOS stores task entries bit0-clear and relies on forced Thumb). **No tail-chaining in v1** (mod.rs:386 — the run loop delivers the next pending exception on the next iteration); **no double-fault lockup** (a HardFault inside HardFault re-enters the handler — differs from Bramble’s M0+ lockup at `PC=0xFFFFFFFF`, documented here so ports don’t assume it).

Delivery (all must hold — Bramble §5.5 table, same semantics): IRQ enabled in ISER; pending in ISPR; PRIMASK==0 (NMI/HardFault exempt — `can_fire`); priority < current (BASEPRI/s-stack/0xFF); strictly higher-priority nesting only (same-priority never nests); FAULTMASK reads 0 (stub — MRS SYSm 17/18 returns 0, no masking). Priority model: Reset -3 / NMI -2 / HardFault -1 fixed; SVCcall/PendSV/ SysTick via SHPR; IRQ 0-96 via IPR (256 levels, byte per IRQ — M3, not M0+'s 2-bit nibbles).

5.3 5.3 Timing model

`cycles += 1` per retired instruction (`mod.rs:473`); DWT retires `1 + flash_latency` (ACR LATENCY, min 2) without pacing the core:

Instruction class	Cycles	Notes
ALU (ADD/SUB/MOV/CMP/AND/...)	1	Single-cycle
LDR/STR (all variants)	1	Interpreter-budget (no 2-cycle memory penalty)
LDM/STM/PUSH/POP	1 + N	N = register count (same formula as Bramble)
BX/BLX	1	No 3-cycle indirect penalty
BL (32-bit)	1	No 4-cycle link penalty
B taken / not-taken	1 / 1	No pipeline-flush modeling
MUL	1	Single-cycle
DWT CYCCNT read	1 + LATENCY	Only counter sees wait states (fixes <code>micros()</code>)

The cycle accumulator converts retired instructions to peripheral time: `timing_tick` is `step_batch` (peripherals read INSTRUCTION_COUNT deltas); SysTick counts raw budget units (ARM spec); `timer_tick/rtc_tick/systick_tick` run once per batch, not per instruction (~100K× cheaper, 3.8× speedup).

5.4 5.4 CPU state reference

Regs `r[16]`, `xpsr`, `primask`, `control`, `msp`, `psp`; Cpu `regs`, `cycles`, `fault`, `it_*`, `ipshr`, `exc_stack`, `deliver_irqs`, `sleeping`, `dsp`, `exclusive`; CpuFault `pc`, `op1`, `op2`, `len`.
Rust API: `new/reset/sp/read_msp/read_psp/write_msp/write_psp/ take_exception/exception_return`.
JS surface (§8): `getRegisters/getPc/getSp/setReg/setPc/takeFault/memRead32/ memWriteBytes`.

6 Part 6: Peripherals and Virtual Devices

Register-level models (Full unless noted); the complete flag/IRQ/DMA truth table is `docs/PERIPHERALS.md` (SVD census 41 Full, 1 Partial-RCC, 0 stubs). Each subsection below follows the same shape as a Bramble §4.3 row, expanded: base address + IRQ + register table + behavior + events + DMA channels + host-side virtual devices.

6.1 6.1 GPIO

Bases: GPIOA 0x40010800, GPIOB 0x40010C00, GPIOC 0x40011000, GPIOD 0x40011400, GPIOE 0x40011800 (HD/CL).

Offset	Register	Notes
0x00	CRL	Port config low (pins 0–7): MODE[1:0] + CNF[1:0] per pin
0x04	CRH	Port config high (pins 8–15)
0x08	IDR	Input data (read-only; electrical model + slew, see below)
0x0C	ODR	Output data (also pull select for CNF=01 inputs)
0x10	BSRR	Bit set/reset (low half sets, high half resets)
0x14	BRR	Bit reset
0x18	LCKR	Lock sequence + nibble freeze until reset

Electrical model (`pin_level()`): input pull-up/down (CNF=01, ODR bit selects direction), floating input (external driver or 0), push-pull output readback (slew-aware), open-drain (low driven; high released → external pull or 0), external drivers (JS read callbacks) win over driven state, analog → 0. `GPIO_SLEW` + `gpio_set_slew(n)`: output transitions land in `pending_transitions`; IDR shows the old level until settle. ODR/BSRR/BRR write the full register (inputs use ODR for pull selection); output-mode side effects (device callbacks, EXTI) fire for output pins only. LCKR 3-write sequence freezes nibbles. AFIO SWJ_CFG reserves PA13–15/PB3–4 per debug mode. Pin events: flat [`port,pin,level`] on NEW driven levels (cap 1024, drop-wholesale), `takePinEvents()/clearPinEvents()`. Fan-out: EXTI edges, PC13 tamper, PA0 WKUP edge, SPI CS callbacks, ADC analog wires.

6.2 6.2 USART

Bases: USART1 0x40013800 (IRQ37), USART2 0x40004400 (IRQ38), USART3 0x40004800 (IRQ39), UART4 0x40004C00 (IRQ52), UART5 0x40005000 (IRQ53) (+ USART6/7/8 slots for HD superset: IRQ71/82/83).

Offset	Register	Notes
0x00	SR	TXE(7)/TC(6)/RXNE(5)/ORE(3)/FE(1)/LBD(8); <code>sr_read_armed</code> clear path
0x04	DR	TX pushes <code>UartTx</code> + TX FIFO; RX pops FIFO (clears RXNE/ORE/LBD/FE)
0x08	BRR	Baud divisor; <code>byte_time</code> = <code>BRR*10</code> instr (else 6250)
0x0C	CR1	SBK(0)/TCIE(6)/TXEIE(7)/RXNEIE(5), UE/TE/RE
0x10	CR2	LBDIE(6)/LINEN(14)
0x14	CR3	HDSEL(3) loopback/IREN(1)/EIE(0)/DMAT(7)/DMAR(6)
0x18	GTPR	Guard time / prescaler (stored)

TXE byte-time pacing with IRQ spacing (`txe_clear_until` = `now` + `byte_time`); TXE immediate for polling, IRQ spaced (TXEIE storm note: STM32duino `Transmit_IT` wedges without the arm). RX 16-deep FIFO, overflow → ORE + RXNE + DMA-RX. ORE clears on SR-read + DR-read (RM0008 sequence via `sr_read_armed`); sticky-ORE wedged UART RX permanently before the fix. LIN: SBK self-clears per batch; LBD in LIN mode else FE + 0x00 byte (`uart_inject_break`). Loopback = HDSEL bit 3 only (IRLP must not trigger it — old 1<2 bug). DMA: USART1 TX ch4/RX ch5, USART2 6/7, USART3 2/3, UART4 11/12, UART5 12/13. Bootloader (AN3155) claims USART1 RX while enabled. IDLE/PE/NE/CTS never set (benign: no error injection).

6.3 6.3 SPI

Bases: SPI1 0x40013000 (IRQ35), SPI2 0x40003800 (IRQ36), SPI3 0x40003C00 (IRQ51).

Offset	Register	Notes
0x00	CR1	DFF(11) 16-bit, CRCEN(13)/CRCNEXT(12), CPOL/CPHA, order, SPE
0x04	CR2	TXEIE/RXNEIE, FRF(4) TI frame, DMA TX(1)/RX(0)
0x08	SR	TXE(1)/RXNE(0)/CRCERR(4); OVR/MODF/BSY never (sync)
0x0C	DR	Synchronous transfer to ext device / MISO inject / <code>SpiTransfer</code>
0x10	CRCPR	CRC polynomial (reset 7)
0x14/18	RXCRCR/TXCRCR	Running CRC state
0x1C/20	I2SCFGR/I2SPR	I2S mode (triangle <code>generate_i2s_audio</code> stub)

NSS hardware output (SSOE): PA4/PB12/PA15 driven via `gpio_exti_trigger`. Active device = CS-low device else first. 16-bit mode transfers MSB-first as two bytes; `spi_take_miso` inject overrides device bytes. CRC MSB-first (8-bit poly&FF/init FF, 16-bit init FFFF); NEXT phase clocks TXCRC and compares RXCRC → CRCERR (DR read clears). FRF TI mode: explicit data path identical, NSS phasing only. DMA: SPI1 TX ch3/RX ch2, SPI2 5/4, SPI3 10/11. Devices: `SpiFlash` (JEDEC/RDID/status/program/erase, `add_spi_flash`), Lcd 128×64 mono (`add_lcd`, `lcdFb`), Touchscreen ADS7846 deferred-reply (`add_touchscreen`, `touchscreen_set_touch`), `UsartProbe` log tap, software bit-bang SPI (`add_software_spi`, mode-0 rising-edge only).

6.4 6.4 I2C

Bases: I2C1 0x40005400 (EV IRQ31/ER IRQ32), I2C2 0x40005800 (EV IRQ33/ER IRQ34) (+ I2C3 slot: EV/ER IRQ72/73).

Offset	Register	Notes
0x00	CR1	START/STOP/ACK/PEC/POS/ALERT/SWRST/PE
0x04	CR2	FREQ + ITEVTEN(9)/ITBUFEN(10)/ITERREN(8) + DMAEN
0x08	OAR1	ADD[9:0] + ADDMODE(15) 7/10-bit
0x0C	OAR2	Dual address + ENDUAL
0x10	DR	TX pushes device/ <code>I2cWrite</code> , RX pops/ <code>I2cRead</code> ; R-bit hook
0x14	SR1	SB/ADDR/BTF/RXNE/TXE/AF/STOPF/SMBALERT/PECERR
0x18	SR2	MSL/BUSY/TRA/G ENCALL/DUALF
0x1C	CCR	Clock control
0x20	TRISE	Rise time
0x30	PECR	Packet error code

Master: Idle → StartSent (SB+BUSY+MSL on START) → AddrSent (ADDR or AF+NACK+auto-STOP) → Active (TXE/RXNE+TRA+DMA). DR read clears RXNE+BTF and pulls the next device/`i2c_take_rx` byte; DR write pushes and clears BTF. SR1 read arms ADDR/STOPF; SR2 read clears ADDR. STOP preserves a pending RXNE byte (single-byte NACK+STOP path) else resets. BTF set when ITBUFEN clears mid-transfer, cleared on DR read/write (stuck-BTF wedged EV re-pends before the fix). AF set-on-NACK, cleared on next START/reset. Slave (`i2c_inject_*`): OAR1 7/10-bit + OAR2 dual + general-call match, ADDR/STOPF sequences,

RXNE/TXE + EV IRQs, no slave DMA. PEC: CRC-8 poly 0x07 while PECEN; PEC bit sends/-compares. SMBus ALERT: CR1.13 edge → I2cAlert event (out), inject → SR1.15 + ERR IRQ (in, W1C). DMA: I2C1 TX ch4/RX ch5 (ch6/7 on AFIO MAPR remap). Devices: I2cEeprom (add_i2c_eeprom), I2cOled SSD1306 (add_i2c_oled, i2cOledFb).

6.5 6.5 TIM

Bases: TIM1 0x40012C00 (IRQ24), TIM2 0x40000000 (IRQ28), TIM3 0x40000400 (IRQ29), TIM4 0x40000800 (IRQ30), TIM5 0x40000C00 (IRQ50), TIM6 0x40001000 (IRQ54), TIM7 0x40001400 (IRQ55) (+ TIM8–14 slots for HD superset: IRQ70/20/25/26/43/54/51).

Offset	Register	Notes
0x00	CR1	CEN, DIR/CMS, ARR preload; mask 0xFFFFEF17F
0x04	CR2	MMS (010 = TRGO on update)
0x08	SMCR	Slave SMS/TS (gated/trigger/reset/extclk/encoder)
0x0C	DIER	UIE/CCxIE/BIE/TIE/UDE/CCxDE (UIE → enable_irq)
0x10	SR	UIF/CCxIF/BIF — W0C (sr &= value)
0x14	EGR	UG → generate_update
0x18/1C	CCMR1/2	CCxS: 0 = output, ≠0 = input-capture source
0x20	CCER	CCxE/P/NP polarity + enable
0x24	CNT	Fast-path read (no advance); 16-bit
0x28	PSC	Prescaler (PSC+1 divisor)
0x2C	ARR	Reset 0xFFFF (RM0008 — old 0xFFFFFFFF wedged catch-up)
0x30	RCR	Repetition counter
0x34–40	CCR1–4	Compare/capture (input-capture latch target)
0x44	BDTR	TIM1/8 only: DTG/LOCK/BKE/BKP/AOE/MOE
0x48	DCR	DBA/DBL burst window
0x4C	DMAR	Burst port (routed DBA+idx, wraps every DBL+1)

Closed-form advance jumps to event ticks (update wrap + CC matches), bit-identical to per-tick simulation (1409 ms → 11 ms, 124×). Slave modes via itr_master (TIM2←TIM1, TIM3←TIM2/TIM1, TIM4←TIM3) + CEN check; encoder via TI1/TI2 + DIR. Dead-time DTG decode narrows pwm_duty = CCR*100/(ARR+1). Break: TIM1 BKIN PB12 vs BKP (BKE+MOE → clear MOE+BIF+BIE; AOE re-arms). Input capture per batch: CCxS → source pin via AFIO remap + tim_chan_pin (TIM2/3/4 remaps), polarity CXP/CXNP, ICPSC, PWM-input both-edges → period/pulse + TimCapture + IRQ. TRGO (MMS=010) + CC triggers fan out to adc_timer_trigger. tick_frozen/rebase move last_tick only (no catch-up wedge).

6.6 6.6 ADC

Bases: ADC1 0x40012400 (IRQ18), ADC2 0x40012800, ADC3 0x40013C00 (HD; shared IRQ path).

Offset	Register	Notes
0x00	SR	AWD(0)/EOC(1)/JEOC(2)/JSTRT(3)/STRT(4); direct-assign = value & 0x3F
0x04	CR1	AWDIE(6)/JEOCIE(7)/EOCIE(5)/AWDEN/DISCEN/DISCNUM/JAUTO/DUAL
0x08	CR2	ADON/DMA(8)/EOCS(10)/CONT(16)/EXTSEL(17–19)/EXTTRIG(20)/SWST

Offset	Register	Notes
0x0C/10	SMPR1/2	SMP codes 0–7 → 14/20/26/41/54/68/84/252 cycles
0x14–20	JOFR1–4	Injected offsets
0x24/28	HTR/LTR	Analog watchdog thresholds
0x2C/30/34	SQR1/2/3	Regular sequence (len at SQR1[23:20] — old [19:16] bug ran len-1)
0x38	JSQR	Injected sequence
0x3C–48	JDATA1–4	Injected results
0x4C	DR	Regular result (read clears EOC; DUALMOD packs ADC2:ADC1)

Conversion state machine (`Convend_at,pos,len,cycles`): $T_{conv} = SMP + 12.5$ cycles at 1 instr = 1 cycle; scheduled on SWSTART/JSWSTART/timer/EXTI/CONT/JAUTO/discontinuous resume. EOC per conversion unless EOCS (sequence-end only); STRT/JSTRT at `pos==0`; AWD vs HTR/LTR gated on CR1.0; DISCNUM chunks suspend; DUALMOD==6 on ADC1 fans out lockstep slave start/force-complete. Sources (priority): wired GPIO analog (`gpioSetAnalog`) > DAC output (PA4/PA5) > nominal internal (temp 0x6EE, VREF 0x5D2, VBAT 0xC7F, host-overridable via `adc_set_internal`) > sim value; real sources sample through RC $V(t)$ with `adcSetRcTau`. External triggers: EXTSEL/JEXTSEL tables (TIM1_CC1/2/3, TIM2_CC2, TIM3_TRGO, TIM4_CC4, EXTI11 / injected TIM1_TRGO/CC4, TIM2_TRGO/CC2, TIM3_CC4, TIM4_TRGO, EXTI15), new conversion only when idle. DMA: ADC1→ch1, ADC2→ch2, ADC3→DMA2ch5. `rebase_clock` aborts in-flight conv/jconv (stale `end_at` completed instantly post-NRST before the fix).

6.7 6.7 DAC

Offset	Register	Notes
0x00	CR	EN1(0)/EN2(4) gates (<code>dma_request(3/4)</code> use EN bits as 1)
0x08/0C/10	DHR12R1/DHR12L1/DHR8R1	CH1 holding (12R &0xFFF, 12L >4, 8R <4)
0x14/18/1C	DHR12R2/DHR12L2/DHR8R2	CH2 holding
0x20/24/28	DHR12RD/DHR12LD/DHR8RD	Dual holding (splits low/high half)
0x2C/30	DOR1/DOR2	Readout (updated immediately on any DHR write)
0x34	SR	Stored only

Enabled channels drive PA4/PA5 analog wires consulted by `ADC::channel_voltage()`; `DacWritechan,value` per channel (dual emits two). No trigger/triangle/noise/wave, no timing.

6.8 6.8 DMA

Offset	Register	Notes
0x00	ISR	GIF/TCIF/HTIF/TEIF per channel nibble; $TCIF_N = (N-1)*4+1$ (real 1)
0x04	IFCR	Write-1-clears per 4-bit nibble
per-ch 0x08+ch*0x14	CCR	EN(0)/TCIE(1)/HTIE(2)/TEIE(3)/DIR(4)/CIRC(5)/PINC(6)/MINC(7)/
+0x04	CNDTR	Count; latched to <code>ndtr_init</code> at EN rising edge (CIRC reloads it)
+0x08	CPAR	Peripheral address
+0x0C	CMAR	Memory address

DIR=1 mem→periph (src CMAR, push to CPAR), DIR=0 periph→mem (absorb CPAR, write CMAR), M2M (CR bit 14) CPAR→CMAR memcpy; size = `max(psize,msize)*NDTR` (8-bit SPI needs PSIZE/MSIZE=00 — 16-bit pushes 2× the wire bytes). Global streams DMA1 ch0–6 → 0–6, DMA2 ch0–4 → 7–11 (shared completion bitspace/IRQ table — local indices collided DMA1-CH4 with DMA2-CH4 before). `dma_request(ch)` queues; `tick()` executes when `EN&&NDTR>0`, then drains only its own streams (`dma_take_completions_masked`). CIRC: TCIF+HTIF set, NDTR reloads, EN stays (continuous); else TCIF, EN+NDTR cleared. EN write triggers immediate `do_xfer + set_dma_intr_info(stream,irq,flags)` with TCIE=bit1/HTIE=bit2/TIE=bit3 (old 4/3/2 decode fired IRQs on DIR). IRQs DMA1 11–17, DMA2 56–60. Whole pump in Rust (`rustcpu_dma_pump()` builds the op plan and executes against Rust RAM, zero JS; completion signaled LAST so TC IRQs fire after data). See §8.4 for the JS surface.

6.9 6.9 CAN

Offset	Register	Notes
0x000	MCR	Mask 0x180FF (INRQ..TTCM + RESET + DBF; .
0x004	MSR	INRQ/SLEEP state
0x008	TSR	CODE/TME/RQCP; old TSR.24–26 level-TX arm
0x00C/010	RF0R/RF1R	RX FIFO status (FMP<2 store else FOVR; RFOM
0x014	IER	FMPIE/FFIE/FOVIE + ERRIE + TMEIE (rising
0x018	ESR	EWG/EPV/BOF/LEC error flags
0x01C	BTR	SILM/LBKM (LBKM loops back through filters; L
0x180–1AC	TX0–2	Mailboxes (TXRQ=TIR.0 → CODE/TME/RQCP;
0x1B0–1CC	RX0–1	FIFOs, depth 2
0x200+	FMR/FM1R/FS1R/FFA1R/FA1R/F0–55	Filters: 16b×2 mask/list, 32b mask/list; FINIT=1

Identifier: `tir»21` for STD and EXT (simplified). TX IRQs edge-triggered (pend once on TXRQ submit + on TMEIE-rising-with- completion-latched; W1C clears); RX/SCE level-gated. Silicon: all 28 banks live in CAN1; CAN2 filter reads 0, writes ignored, match delegated via `can_match_can2()`. Events `CanTx/CanRxcn,id,len, data[8]` (11b/29b by IDE). Firmware filter bank 0 is ID-list mode → inject ID 0 (the periph39 autopilot).

6.10 6.10 RTC / BKP / PWR / FLASH

RTC base 0x40002800 (IRQ3). CRH 0x00 SECIE(0)/ALRIE(1)/OWIE(2) — RM0008 order (old SECIE/ALRIE swap fixed); CRL 0x04 SECF(0)/ALRF(1)/OWF(2) W0C, RSF(3), CNF(4), RTOFF(5); PRLH/PRL 0x08/0C (PRL=1M → 1 CNT/sec); DIVH/DIVL/CNTH/CNTL 0x10–1C (writes update `cnt+last_cnt`); ALRH/ALRL 0x20/24 (edge `last_cnt<alarm<=cnt`). `tick()` gated on RTOFF; `RtcAlarmalarm + IRQ3` on alarm/second/wrap; `rebase_clock` moves `last_tick`.

BKP base 0x40006C00 (RM0008 layout): DR1–10 0x04–0x28 (u16), RTCCR 0x2C (&0x3FF), CR 0x30 (&0x03: TPE.0+TPAL.1), CSR 0x34 (TEF.8+TIF.9, W1C via CTEF.0/CTI.1). `bkp_tamper(rising)` from PC13 input edges: active level by TPAL (0=high,1=low) → clear all DR, set TEF+TIF, pend TAMPER IRQ2. Output-driven PC13 LED does not tamper.

PWR base 0x40007000: CR 0x00 (&0x1FF: PDDS.1/LPDS.0/PVDE.4/ PLS.7:5), CSR 0x04 (WUF.0 RO-except-clear, PVDO.2 RO, EWUP.8 RW). Supply model `supply_mv=3300`, `PVD_MV=[2200..2900]` thresholds; `pvd0 = PVDE && supply<thr`; CR-write/`pwr_set_supply_mv` edges fan out via `exti_line_edge(16)` `wkup_edge` (rising+EWUP) latches WUF. Trait: `pwr_standby_selected/pwr_mode/pwr_estimate`.

FLASH base 0x40022000: ACR 0x00 (&0xFF, `flash_latency()`=ACR&7 feeds DWT), KEYR 0x04 (KEY1 0x45670123→KEY2 0xCDEF89AB clears LOCK.7), SR 0x08, CR 0x0C (&0x3FFF,

STRT.6 rising: WRPR bit (AR>12)&0xF → WRPR.TERR.4 else BSY.0), AR 0x10, OPTKEYR 0x14, OBR 0x1C (USER[9:2] settable, flash_wdg_hw()=OBR.2==0), WRPR 0x20. Guest flash contents immutable (BSY-while-PG is correct silicon — DFU firmware must not poll it).

6.11 6.11 FSMC / SDIO

FSMC data: NE1-4 at 0x60000000/04000000/08000000/0C000000 (16M each), NAND2/3 at 0x10000000/0x20000000, PC-CARD at 0x30000000; regs at 0x40000000+: BCR/BTR×4, PCR/PMEM/PAT, ECCR2 0xB4, ECCR3 0xB8, BWTR×4 0x104-0x11C. NOR requires MBKEN.0 (read) + WREN.1 (write); NAND/PC always enabled; read_sized/write_sized byte-assemble per access width. NAND ECC: Linux-nand_ecc row+column Hamming while PCR.ECCEN.6, ECCPS row depth, single-bit-locatable syndrome. FsmcAccessbank:1-7,offset,write,size,value on every data access (backing or not). Backing: FsmcNor (add_fsmc_bank, FSMC.BANK1..7) + Display parallel LCD (0x2A/0x2B windows, 0x2C draw, canned replies); direct fsmcWriteByte/fsmcReadByte. No timing/wait states; missing backing reads 0.

SDIO base 0x40018000 (IRQ49): POWER 0x00 / CLKCR 0x04 / ARG 0x08 / CMD 0x0C (CPSMEN.10 exec, idx 5:0) / RESPCMD / RESP0-3 / DTIMER / DLEN / DCTRL / DCOUNT / STA (CMDREND6/CMDSENT7/DATAEND8/DBCKEND10/CTIMEOUT2 + dynamic RXACT/TXACT/FIFO flags) / ICR W1C / MASK / FIFOcnt / FIFO 0x80 (32 words). Synchronous exec_cmd: no-card → CMDSENT/CTIMEOUT; CMD0 reset, CMD1 MMC OP_COND busy-3, CMD2/3 CID/R6, CMD8 R7 / MMC EXT_CSD, CMD9 CSD (v1 if SDSC byte mode), CMD16 BLOCKLEN, CMD32/33/35/36/38 erase → 0xFF, CMD17/18 READ, CMD24/25 WRITE, CMD55 APP + ACMD41 (busy-3, HCS→byte/block addressing). Empty DLEN completes immediately. DMA DCTRL.3 → dma_request(11) (DMA2 CH4); polled + DMA share one path. Backing: SdCard (add_sd_card("SDIO",data): sectors, OCR, EXT_CSD, CID/CSD v1+v2, R6, block R/W).

6.12 6.12 USB FS / OTG_FS

USB FS base 0x40005C00 (LP IRQ20/HP IRQ19/WKUP IRQ42): EPnR 0x00-0x1C (EA/KIND/-TYPE, STAT_TX/RX toggle-on-1, DTOG RO, CTR_TX/RX W0C + SETUP retire, ISO-type no-STALL → HP), CNTR 0x40 (FRES.0 holds reset with no event, PDWN.1 gates all, FSUSP.3 forces SUSP / clear wakes WKUP42, RESUME.4 one frame, CTRM/RESETM/-SOFM/ESOFM/SUSPM/WKUPM), ISTR 0x44 W0C (CTR/DIR/EP_ID derived), FNR 0x48 (frame+RXDP), DADDR, BTABLE &FFF8, PMA 0x400-0x7FF 1024 B sparse (PMA_ACCESS=2: word W at 2W 4B-stride; desc stride 16/ep: DESC0 TX +0/+4, DESC1 RX +8/+12; DB-bulk KIND+bulk ping-pong DTOG, stays VALID first fill). SOF 1 ms engine (FNR/RXDP/SUSP/WKUP; SOF = bus activity — idle auto-suspend removed; suspend is FSUSP-forced only). usb_reset (FRES / bus SE0; FRES release is NOT a reset — only usb_bus_reset SE0 + reattach), detach (tokens stop, VALID sticks, SOF freezes, ESOF immediate), complete_in (→VALID+CTR-clear, powered + attached → UsbInep,data, NAK, DTOG), deliver_rx (VALID required except SETUP always ACK even NAK — ST never re-arms after status-IN; DADDR filter, data_fits, NAK+CTR+DTOG). EPnR CTR write-1-no-effect preserves the opposite completion (CDC 2-packet bug fix). STM32duino CDC is EP1-OUT/EP2-IN/EP3-CMD.

OTG_FS base 0x50000000 (IRQ67, F105 only): global GOTGCTL/GOTGINT/ GAHBCFG/GUSBCFG/GRSTCTL (CSRST/RXFFLSH/TXFFLSH/AHBIDL)/ GINTSTS+GINTMSK (W1C)/GRXSTSR(peek)/GRXSTSP(pop)/GRXFSIZ/GNPTXFSIZ/ GNPTXSTS/GCCFG (PWRDWN gates)/CID/HPTXFSIZ/DIEPTXF1; device DCFG (DAD)/DCTL (RWUSIG/SDIS/SGONAK/CGONAK)/DSTS/DIEPMSK/DOEPMSK/DAINT/DAINTMSK/DVBUSDIS/PCGCCTL + IN 0x900+n*0x20/0x800+n*0x20 (CTL/INT W1C/TSIZ) ×4; host HCFG/HFIR/HFNUM/HPTXSTS/HAINT/HAINTMSK/ HPRT (PCSTS/PCDET/PENA/PRST/PPWR) + HC 0x500+ch*0x20 (CHAR/SPLT/INT W1C/TSK) ×8; FIFOs 0x1000+n*0x1000 word-only. Device: powered=!PWRDWN, link_down=detached||sdis,

RXFLVL-derived level IRQ, `xfrsiz` saturating, SOF 72000 instr, `core_reset() = fresh()` preserving `last_tick+host_attached` (PHY stays), `bus_reset` (USBRST+ENUMDNE, reattach), IN completes when pushed bytes reach XFRSIZ (ST stages TSIZ+EPENA *before* pushing — zero-length when XFRSIZ==0 with PKTCNT). Host: CHDIS→CHHLT+HCHALTED, CHENA rising (+ HostRx for IN), HostTxch, ep, setup, data on pushed≥XFRSIZ, `host_feed_in` (RX-FIFO + DONE BCNT0, TSIZ-=len, XFRC+ACK). `otg_bus_reset/detach/inject_setup/inject_out/host_attach` (+JS + .d.ts). One shared instance owns regs+FIFOs (SVD splits GLOBAL/HOST/DEVICE/PWRCLK — `from_svd` registers once).

Every peripheral above also funnels through the Wokwi-style event queue where it has one (`UartTx/SpiTransfer/I2c*/ExtiEdge/AdcDone/ TimUpdate/TimCapture/DacWrite/CrcResult/RtcAlarm/V CanRx/FsmcAccess/UsbIn/I2cAlert/HostTx/HostRx/ItmByte` — drained per batch by `STM32F1._drain_event` into the §8.2 callbacks); DMA itself exposes registers, not events (§8.4).

6.13 6.13 NVIC / SysTick / SCB / MPU / Debug

NVIC window 0xE000E100-0xE500: ISER 0x00-0x1C (newly-enabled + pending promotes), ICER 0x80-0x9C, ISPR 0x100-0x11C, ICPR 0x180-0x19C (0x280 RESERVED-ignored, not an ICPR alias — old mis-alias fixed), IABR 0x200-0x21C RO (writes ignored; `clear_active_bit()` on return cures the phantom-active leak), IPR bytes 0x200-0x2FF (via 0xE000E300) + words 0x300-0x3EF. IRQ_COUNT 97, pending: u128 (OFFSET 16+irq), priority[97], `sys_handler_priority[16]=0`, `active_prio_stack`, `last_popped` (hot re-pended IRQ yields to another pending — TXE can't starve EXTI13 within the 64-take `intr_next` budget). `can_fire`: PRIMASK blocks except NMI/HardFault, enable-gated ext, prio < current (BASEPRI/stack/0xFF). Fixed NMI 0/HardFault 1, else SHPR (exc 4.15) else IPR else 0xFF. SysTick debt: `elapsed/period` (min 16), phase-preserving `last_trigger+=ticks*period`, `debt=ticks-1` (or ticks if already pending) cap 16; `systick_take`: `debt- + re-pond` (exactly-once per return — whole-debt drains coalesced and lost ticks before the fix). SysTick regs: CSR 0x00 (ENABLE/ TICKINT/CLKSOURCE/-COUNTFLAG), RVR 0x04, CVR 0x08 (write clears + re-anchors), CALIB 0x0C=0. Constants: NMI -14/HARD -13/MEM -12/BUS -11/USAGE -10/SVC -5/PENDSV -2/SYSTICK -1.

SCB window 0xE000ED00-0xEDFF: CUID 0x00 (0x411CC230), ICSR 0x04 (PENDSVSET.28/PENDSTSET CLR.27/25, VECTPENDING), VTOR 0x08 (&FC00), AIRCR 0x0C (KEY 0x05FA, SYSRESE-TREQ → watchdog reset), SCR 0x10 (SLEEPDEEP.2 → deep sleep), CCR, SHPR1-3 0x18-0x20 → NVIC prio, SHCSR 0x24 (BUSFAULTENA.18/USGFAULTENA.16), CFSR 0x28 W1C (+MPU MMFSR), HFSR/DFSR/MMFSR/BFAR/AFSR/CPACR, MPU TYPE 0x90 (0x0800)/CTRL 0x94/RNR 0x98/RBAR/RASR 0x9C/0xA0 + aliases (VALID latches RNR), debug 0xF0-0xFC → `swd::scb_debug*`. `raise_fault(kind,addr)`: 0/1/2 → IBUSERR/PRECIERR+BFAR+VALID → BusFault if enabled else HardFault+FORCED; 3 → UNDEFINSTR → UsageFault else HardFault.

MPU: 8 regions RNR/VALID/A1-3/prio/subregions/AP (111==110 RO)/ XN/background/PPB-priv+XN; deny read-0/drop + MMFSR/MMFSR + MemManage (or HardFault), CFSR W1C; exec-deny loud halt; priv published on MSR/exception, DMA raw trusted, stacking bypasses. Off fast path: plain-static MPU_ON + #[cold] outlines + raw fetch.

Debug slice (`swd.rs`, transaction-level — no pin/clock modeling): SWD DPv1 (DPIDR 0x2BA01477, CTRL/STAT ACKs + sticky W1C, SELECT, RDBUFF) + MEM-AP (CSW 0x23000052 SIZE/AddrInc, TAR auto-inc, DRW, BD0-3/CFG/BASE/IDR) + Cortex debug (DHCSR DBGKEY/C_HALT DCRSR/DCRDR incl. MSP/PSP, DEMCR TRCENA + VC_HARDERR halt) routed from the SCB window 0xF0-0xFC on both maps + 4 exact-range watchpoints (halt-after-access, first-trip latch) + minimal JTAG TAP (IDCODE 0x4BA00477, BYPASS/DPACC/APACC/ABORT). Hot path: plain-static DEBUG_HALT (run) + WATCH_ON (guest data only).

DWT window: CTRL 0x00, CYCCNT 0x04 (cycles += $\Delta\text{instr} \times (1+\text{flash_latency}$, min 2); write sets offset; fixes `micros()/recoverBus` spin). ITM: STIM 0x000-0x07C (port 0 only), TER 0xE00, TPR 0xE40, TCR 0xE80, LAR 0xFB0; `ItmByteport:0`, byte when TCR.ITMENA

+ TER[0].

RCC base 0x40021000: CR 0x00 (HSEON.16→HSERDY.17 mirror, PLLON.24→PLLRDY.25, HSION forced), CFGR 0x04 (SWS follows SW), CIR 0x08 (CSSF.7, CSSC.23 clears), APB2RSTR/APB1RSTR/APB1ENR, BDCR 0x20 (LSERDY mirrors, BDRST clears), CSR 0x24 (RMVF.24 → 0x0C). `sysclk_hz()` follows SWS (PLL $\text{fin} \times \text{mul}$, HSE/2, $\text{mul} \geq 14 \rightarrow 16$), HSI/HSE 8 MHz, `clocks_hz()` (sys/hclk/pclk1/ pclk2), `mco_hz()` (SYS/HSI/HSE/PLL÷2), `fail_hse()` (clear HSERDY; CS-SON → CSSF+SWS=HSI+NMI -14), `wake_from_stop()` (SWS=00, SW kept). Timing deliberately unscaled.

IWDG base 0x40003000: KR 0x00 (0x5555 unlock, 0xAAAA reload, 0xCCCC enable+reload), PR 0x04 (3b), RLR 0x08 (12b), SR 0x0C (read-destructive). `tick_instructions = 128 \times \text{div} (div 4–256); running = enabled || flash_wdg_hw() (OBR WDG_SW); free-run tick() + rebase_clock (NRST re-anchor). WdogReset1 + reset request on expiry; SR busy clears after one period.`

WWDG base 0x40002C00: CR 0x00 (WDGA.7, T6:0, reset 0x7F), CFR 0x04 (EWI.9 0x200, TB presc 1/2/4/8, W6:0), SR 0x08 (EWIF.0, W0C). `tick_inst = 256 \times \text{presc}`; free-run `tick()` + `rebase`; underflow clamps 0x3F; crossing 0x3F → SR + EWI (IRQ0 if CFR.9) + `WdogReset2` + reset if WDGA; CR-write window check (old>T while WDGA && window≠0 → immediate reset) + `refresh(last_tick=now)`.

CRC base 0x40023000: DR 0x00 (reset 0xFFFFFFFF, write feeds MSB-first poly 0x04C11DB7 ×32), IDR 0x04 (&0xFF), CR 0x08 (bit 0 resets DR). `CrcResultvalue` on DR read. AFIO base 0x40010000: EVCR 0x00, MAPR 0x04, EXTICR1–4 0x08–0x14, MAPR2 0x1C; `remap_status` + `exti_port` + `swj_cfg`. EXTI base 0x40010400: IMR/RTSR/FTSR/SWIER/PR + `exti_line_edge`, `fire_line()`, standby gating (see §6.9 of the previous revision — folded here).

6.14 6.14 IRQ number table

IRQ	Source	IRQ	Source
–14	NMI (RCC CSS)	–1	SysTick (debt model)
–13	HardFault (escalation)	0	WWDG EWI
–12/–11/–10	MemManage/BusFault/UsageFault	1	PVD (EXTI16)
–5	SVCall	2	Tamper (BKP)
–2	PendSV (ICSR)	3	RTC (alarm/second/overflow)
6/7/8/9/10	EXTI0/1/2/3/4	23 / 40	EXTI9_5 / EXTI15_10
11–17	DMA1 CH1–7	56–60	DMA2 CH1–5
18	ADC1/2 (+ ADC3 path)	19–22	CAN1 TX/RX0/RX1/SCE
63+	CAN2 TX/RX0/RX1/SCE (base 63)	20/19/42	USB LP/HP-ISO/WKUP
67	OTG_FS (F105)	42	OTG WKUP (EXTI18)
24	TIM1	28/29/30	TIM2/TIM3/TIM4
50/54/55	TIM5/TIM6/TIM7	70/20/25/26/43/54/51	TIM8–14 (HD slots)
31/32	I2C1 EV/ER	33/34	I2C2 EV/ER
72/73	I2C3 EV/ER (slot)	35/36/51	SPI1/SPI2/SPI3
37/38/39	USART1/2/3	52/53/71/82/83	UART4/5 (+ USART6–8 slo
49	SDIO	54	TIM6-shared (TIM13 shares

DMA request map: ADC1→1/ADC2→2/ADC3→13(DMA2ch5); DAC→3/4; USART1 4/5, USART2 6/7, USART3 2/3, UART4 11/12, UART5 12/13; SPI1 3/2, SPI2 5/4, SPI3 10/11; I2C1 TX4/RX5 (6/7 remapped); TIM per-timer update/CC (TIM1 2/+CC, TIM5–7 DMA2); SDIO→11 (DMA2 CH4).

7 Part 7: Tested Firmware and Test Suite

7.1 7.1 Test suite

Suite group	Command
Unit gate	<code>node tests/test_all.mjs</code>
Firmware gate	<code>node tests/canary.mjs</code>
Full run	<code>echo -n "AB" node pkg/cli.mjs -config=tests/arduino_periph_test/config.yaml</code>
Browser loop	<code>node tests/test_emulator_js.mjs</code>
CPU proofs	<code>cargo test -release -lib cpu::census && python3 tests/census_16.py && python3</code>
Diff fuzz	<code>python3 tests/fuzz_diff.py -cases 200 -seed 1</code>
CoreMark	<code>node tests/test_coremark.mjs</code>
GDB	<code>node tests/test_gdbstub.mjs</code>
Per-board	<code>node tests/test_board_*.mjs</code>
Browser	<code>TMPDIR=~/.tmp-pw npx playwright test tests/test_browser_site.mjs tests/test_b</code>

Batch-boundary timing: peripherals tick between batches, never mid-batch — tests must be async-style (arm once, poll across batches); only `svc` is synchronous. `canRxArmed` resolved from ELF symbols (never hardcoded — a past hardcoded drifted and cost ~4 s/run). 200M cap for the full run; 50M stops mid-print (not a deadlock). A (0x41) is reserved for DMA RX, B for UART RX (`uart_rx_pending()` gate); firmware filter bank 0 is ID-list → CAN inject uses ID 0; periph39 passes **39/39 on all six F103-map chips**.

7.2 7.2 RP2040/RP2350 comparison

Bramble (RP2040/RP2350, C99) and this emulator share the philosophy (register fidelity, unmodified firmware, GDB, scripted I/O) but differ in mechanics worth knowing when reading both manuals:

Topic	Bramble (RP2040/RP2350)	This emulator (STM32F1)
Host language	C99 + POSIX (pthreads/sockets/mmap)	Rust → WASM + JS ESM
CPU	M0+/M33 Thumb + Hazard3 RV32, icache + JIT	M3 Thumb-2 interpreter, no JIT (int)
Cores	Dual-core (round-robin or threaded big-lock)	Single core + MSP/PSP banks (RTC)
Memory	264K/520K SRAM banks, XIP flash + aliases	Per-chip flash/RAM, FSMC windows
DMA	Synchronous immediate transfers	Batched Rust pump, completion-last
Interrupts	Per-core NVIC, tail-chain/late-arrival	NVIC + 64-IRQ budget + fairness +
USB	Host-enumeration simulation + CDC bridge	FS device + OTG_FS device/host, s
Storage	Flash persistence + FUSE + SD/eMMC + FAT	Flash/SDHC images (no FUSE/FAT)
Network	UART-TCP, TAP, WiFi gSPI, VNet mesh	WS bridge only (no guest TCP/IP)
Debug	GDB 16+16 slots, conditional BPs, dual-core threads	GDB Z0–Z4, 4 watchpoints, SWD/J
Output	Firmware → stdout, diagnostics → stderr	Firmware → <code>getUartOutput()</code> , diag

7.3 7.3 Demo firmwares

Firmware	What it proves	Suite
periph39 (arduino_periph_test)	24 peripherals, 39 checks	canary.mjs + 200M CLI
blink/echo/comprehensive	Bring-up, UART, legacy 10-check	browser + test_emulator
fade/timer_uart/adc_uart	PWM, TIM IRQ, ADC polling	headless suites
pwm_wave/servo/dac_sine	TIM DMA-burst, 50 Hz servo, DAC→ADC	headless + page
rtc_clock/stopwatch	RTC seconds, EXTI + TIM2	headless + page
flash_demo/showcase/ws2812	SPI flash, 7-device showcase, DMA strip	headless + page
i2c_scan/i2c_slave	Wire probe, slave @0x42	headless + i2cCard
can_chat/can_dual	LBKM self-talk, F105 dual-CAN	headless + page
mini_rtos/sd_logger	PendSV tasks, SDIO logging	headless + page
usb_cdc/usb_serial/otg	FS enum/echo, real stack, OTG dev/host	headless + page
dfu/coremark	Maple DFU, known-answer CPU	headless + page
board_demo/echo/showcase/rtc	4 targets × matched chips	headless + browser
hd_fsmc	F103RC FSMC + dual DAC	headless + page

7.4 7.4 Known divergence

One firmware-visible workaround remains (both documented in `pkg/cli.mjs` + `pkg/emulator.js`): the I2C model flags I2C1 DR writes with the R-bit, and the driver patches guest RAM $*(0x200002d8)+0x3D$ to `0x22` pre-dispatch so HAL's I2C1 ISR (`hi2c->Mode == 0x22 MASTER_RX`) reads DR. USART probes are log-only taps (no registration needed; config `usart_probe` entries select the stdin UART by name/address).

8 Part 8: JS API Reference

8.1 8.1 Package layout

`stm32f1-emu@3.3.0`, ESM, Node ≥ 18 . Exports: `.` → `pkg/stm32f1.js`, `./emulator` → `pkg/emulator.js`, `./gdb` → `pkg/gdbstub.mjs`, `./wasm` → raw glue, `./cli` → `pkg/cli.mjs`; bins `stm32f1-emu / bluepill-emu`. Files: bridge + wrapper + WASM + GDB + CLI + WS server + `site/board_pins.json` + `svd/`. Raw WASM: 122 exports (do not call directly — `emulator.js` owns `init/batch` policy).

8.2 8.2 High-level API

Factories `create/fromELF/fromBin/fromHex`; `loadELF/loadBin/loadHex/ _reload/reset`; `execute/step/s _drain_events` per batch (discriminants 1–22). `execute()` splits long runs into `getBatchSize()`-chunked `step()`s with the transfer drain after every batch, so each batch's GPIO pin changes land before that batch's transfer callbacks (a CS sampled in `onTransfer` is fresh as of that transfer); `reset/load` paths clear the accumulated per-USART TX buffers too, so no wrapper state survives a reset. `gpio.pin('A'..'G'|0..6, 0..15) → on('change')→unsub`, `read/readInput/setInput/setAnalog`; `usart1/2/3 → onData/send/output (+ USART1 shortcuts uartRx/uartOutput)`; `spi1..3 → onTransfer/injectMiso`; `i2c1..3 → onStart/onWrite/onRead/onStop/injectRx`; `dma1/dma2 (dma[1..2]) → isr/getCcr/getNdtr/getPar/getMar/ setChannel/clearFlags/pending` (see §8.4); `adc1..3 (adc[1..3]) → setVoltage/setCode` and `tim1..7 (tim[1..7]) → enabled/duty/frequency` (see §8.5); top-level `onExtiEdge/onAdcDone/onTimUpdate/onDacWrite/onCrcResult/onRtcAlarm/onWdogReset/onCanTx/onCanRx/onTimCapture/onFsmcAccess/onUsbIn/ onI2cAlert/onHostTx/onHostRx/onPeriphWrite/setSymbols/ resolveSymbol/fsmcWriteByte/fsmcReadByte`; re-exports `parseElf/parseIn`

8.3 8.3 Low-level API

`createEmulator(firmware, flash_size?, ram_size?, vector_table?, svd?, chip?, js_peripherals
uart_addr?, ext_devices?, verbose?, batch_size?)` → `BluepillEmulator` (116 meth-
ods — all of §8.2's underlying calls plus `run/step/stop/close/reset/setBoot0/getBoot0/
boardInfo/getRegisters/getPc/getSp/setReg/setPc/read32/write32/ memRead32/memWriteBytes/take
addJsPeripheral/getBatchSize/getInstCount` and every inject). `CHIPS (8) + chipInfo().
ext_devices: spi_flash/i2c_eeprom/i2c_oled/lcd/touchscreen/ software_spi/fsmc_bank
(name,data|name,size)/sd_card.`

8.4 8.4 DMA API

`mcu.dma1/dma2` (also `mcu.dma[1..2]`); low-level `emu.dmaIsr`, `emu.dmaGetCcr`, `emu.dmaGetNdtr`,
`emu.dmaGetPar`, `emu.dmaGetMar`, `emu.dmaSetChannel`, `emu.dmaClearFlags`, `emu.dmaPending`
+ `raw queue/IRQ` `emu.dmaQueueCount`, `emu.dmaQueuePeek`, `emu.dmaQueueAt`, `emu.dmaPump`,
`emu.dmaTakeAbsorbed`, `emu.dmaAbsorb`, `emu.dmaPush`, `emu.dmaComplete`, `emu.dmaCompleteMany`,
`emu.irqPending`, `emu.irqNext`, `emu.irqReturn`, `emu.irqFinish`; `clocks/power/debug` `emu.rccSysclkHz`,
`emu.rccClocksHz`, `emu.rccMcoHz`, `emu.rccFailHse`, `emu.pwrSetSupplyMv`, `emu.gpioSetSlew`,
`emu.i2cOledWrites`. `DMA1@0x40020000 (7ch) / DMA2@0x40020400 (5ch)` on **every** chip
(builtin + both SVDs), so one surface covers all variants + F105. Channels 1-based; CCR bits
`EN=0 TCIE=1 HTIE=2 TEIE=3 DIR=4 CIRC=5 PINC=6 MINC=7 PSIZE=8-9 MSIZE=10-11 PL=12-13
M2M=14`; `TCIF` at  $(N-1)*4+1$ ; every `irqNext()` pairs with a return. Worked M2M example in
`docs/STM32F1_API.md ("DMA")`.

8.5 8.5 ADC + TIM API

`mcu.adc1..3` (also `mcu.adc[1..3]`) inject the target voltage the converter samples — the host
side of `analogRead`-style firmware: `setVoltage(ch, millivolts)` (0..3300 at `VREF=3.3V`) /
`setCode(ch, code)` (raw 12-bit). Channels 0-15 route to the mapped GPIO pin analog wire
(`PA0-7`, `PB0-1`, `PC0-5` — the exact source the model samples through its RC sample-and-hold),
16-18 (`temp/VREF/VBAT`) use the internal override, higher channels fall back to the global sim
value. Completion is observed via `onAdcDone(adc, chan)`.

`mcu.tim1..7` (also `mcu.tim[1..7]`) observe PWM/servo/LED/buzzer outputs: `duty(ch)` (0-
100 from `CCR/ARR`) + `frequency()` (`PSC/ARR` + live `RCC` tree incl. the `APB x2` rule) —
both 0 unless `CR1 CEN`. Update edges arrive via `onTimUpdate(tim)`.

```
mcu.adc1.setVoltage(0, 1650);    // hold PA0 at mid-scale... await
mcu.execute(1_000_000);
// ...run the firmware, read DR back console.log(mcu.tim3.duty(0),
mcu.tim3.frequency()); // servo PWM readback
```

`execute()` splits long runs into `getBatchSize()`-chunked `step()`s with the transfer-event
drain after every batch, so each batch's GPIO pin changes land before that batch's transfer
callbacks (a `CS` sampled in `onTransfer` is fresh as of that transfer). Batch introspection lives on
the low level: `emu.getBatchSize()` (default 20000) + `emu.getInstCount()` (cumulative).

8.6 8.6 Servers and debug

`serveGdb(...)` → `port,emu,close` (see §3.4); `pkg/cli.mjs` (§3.1–3.3); `pkg/ws-server.mjs`
(§3.5); `site/board_pins.json` (Arduino aliases from STM32duino 2.12.0: `D13=PA5`, `D33=PB1`,
`BUT=PB8`, `D17=PC13`).

9 Part 9: Debugging

9.1 9.1 GDB remote debugging

Setup (mirrors Bramble §9.1 — same two-terminal shape, different binary and default port):

```
# Terminal 1: start emulator with GDB server (default port 1234)
node -e " import('./pkg/gdbstub.mjs').then(async ({ serveGdb }) => {
  const srv = await serveGdb({ firmware, port: 1234, chip: 'gd32f103c8'
    });
  console.log('GDB on', srv.port);
});"
# Terminal 2: connect GDB (ARM only --- single Cortex-M3, no RISC-V
  layout)
arm-none-eabi-gdb firmware.elf -ex 'target remote :3333'
```

Command	Description
?	Halt reason (T05 + thread)
g / G	Read / write all 17 registers (r0–r12, sp, lr, pc, xpsr, LE hex ea
pHEX / PHEX=	Read / write single register by hex number (Pf = PC — decim
m addr,len / M addr,len:data	Read / write memory (hex Maddr,len: lengths; bounds len ≤
c / s (+ vCont;c / vCont;s, c;/s; forms)	Continue / single step (c resumes halts + re-inserts stepped BK
Z0 addr,len / z0 addr,len	Software breakpoint (16-bit BKPT patch, Thumb-masked &~1,
Z2/Z3/Z4 / z2/z3/z4	Write / read / access watchpoints → 4 DWT-style slots (kind r
vCont?	Returns vCont;c;s
Hc/Hg (any)	Accepted (single thread — GDB 15 sends Hc0/Hc1/Hc-1)
qfThreadInfo / qsThreadInfo / qC	m1 / 1 / QC1 (single thread m1, not Bramble's m1,2 dual-core)
qSupported	PacketSize=3fff;QStartNoAckMode+;vContSupported+ (no s
qXfer:features:read:target.xml	Served with offset/length awareness but not advertised (GDE
k / D	OK + close
Ctrl-C (0x03)	Interrupt → S02

Registers: ARM-only single layout (17 regs — no 33-reg RISC-V mode, no dual-core thread selection). Watchpoint stops: T05watch:ADDR / T05rwatch:ADDR / T05awatch:ADDR (halt-after-access — entered as part of a step, like silicon). Limits: BKPT slots unbounded map (addr → orig bytes) + single-step reinsert dance; 4 watchpoint slots; runUntilEvent() polls step(chunk) (default 20000) against swdTakeTrip() (outranks same-batch faults) → swdHalted() → takeFault() (BKPT-hit vs genuine decode-gap S04) → stopped. Proven by tests/test_gdbstub.mjs (35/35) + live GDB-15 sessions; tests/gdb_live_session.sh is dev-only. No qRcmd conditional breakpoints (Bramble-only), no Z1 hardware breakpoints (BKPT covers it), no semihosting intercept (Bramble BKPT 0xAB / EBREAK 0x20026 — not implemented here).

9.2 9.2 Debug output and watch tools

Tool	Call	Meaning
Fault snapshot	takeFault()	One-shot [pc,op] since last call (UNDEFIN
Bus tap	onPeriphWrite(fn)	Every peripheral write fn(addr,width,val
Pin feed	onPinChange(fn)	Chip-driven level changes fn(port,pin,lev

Tool	Call	Meaning
Event queue	<code>drainEvents()</code>	Flat i32 array, 22 discriminants (§8.2)
Symbols	<code>setSymbols(mapText)</code> / <code>resolveSymbol(pc)</code>	Whole-identifier regex; resolve fresh per build
CLI traces	<code>-verbose</code> / <code>-regs</code>	Peripheral traces / register dumps
Batch status	<code>process_batch()</code>	0x80000000 watchdog, 0x40000000 IRQ pending
SWD/JTAG	<code>swd_*/jtag_*</code>	DPv1 (0x2BA01477) + MEM-AP (0x23000000)

10 Part 10: Storage, Networking, Multi-Device

10.1 10.1 Flash images

Firmware loads into the model flash array at init (ELF segments / HEX data / BIN @ base); guest flash stores drop (`write8_raw_unchecked` — DFU stages downloads to a 2 KB RAM buffer resolved from ELF symbols for this reason, while the real unlock/program/BSY sequence still runs). No `-flash storage.bin` write-through file, no sector restore on boot, no save-on-exit (Bramble §10.1 has all three — not implemented here; re-run `createEmulator` for a fresh image).

10.2 10.2 SD card

File-backed SDHC image (`add_sd_card("SDIO", data)`): CID/CSD v1+v2 / OCR / RCA derived, CSD capacity from image size, sector R/W shared with the MMC path (CMD1/EXT_CSD/erase-0xFF, latch-first; HS200/RPMB/ boot partitions out). SDIO host: CMD0/1/2/3/6/7/8/9/12/13/16/17/18/24/25/32/33/35/36/38/55 + ACMD41 (busy-first power-up), 32-word FIFO, DATAEND/D-BCKEND/CMDREND/CMDSENT/CTIMEOUT + MASK-gated IRQ49, DMA2 CH4 shared polled/DMA path, SDSC byte-addressing (HCS=0 → ARG/blocklen, CSD v1, CCS-clear OCR). Bramble's SPI-mode CMD set (CMD0/8/9/10/12/ 13/16/17/18/24/25/55/58 + ACMD41, CSD v2.0, "BRMSD") overlaps on commands but differs on bus (SPI vs SDIO) — port tests, not code. No SPI eMMC (`SdCard` is SD-family only; Bramble CMD1/EXT_CSD-via-CMD8 eMMC "BRMMC" is a documented non-gap), no FUSE mount, no FAT driver (Bramble §10.3–10.4 — out of scope by decision).

10.3 10.3 Networking

`pkg/ws-server.mjs <firmware.elf> [-port=8080] [-max=N]:` serves `site/` over HTTP, streams emulator frames over WS at ~60 fps (`step(20000)` per tick, idle with no clients). Inbound JSON: `uart_rx addr?,byte → uartRxAddr, gpio_set port,pin,high → gpioSetInput, can_inject addr,tir,tdtr,tdlr,tdhr, board_reset → reset() + type:resetDone,bootloader, board_boot0 high → setBoot0`; bad JSON ignored. Outbound `hello firmware` on connect + `e,p,fps,t` frames (`e` = flat events — `Array.from Int32Array` for JSON; `p` = pin triples; 20K batch ≈ 1.1 ms). `site/ws-viewer.html` decodes all 22 event types (UART terminal, 48-pin grid click-to-toggle, event log, FPS/instr, 2 s reconnect). No guest TCP/IP, TAP/NAT, WiFi gSPI, VNet mesh, or multi-instance Unix-socket wiring (Bramble §11 — out of scope by decision; the WS bridge is the closest template for an OpenHW bridge).

10.4 10.4 Multi-device

Page-driven virtual devices through the event queue (FSMC LCD model end-to-end in `tests/test_fsmc_display` — the MCU writes LCD command/data over FSMC BANK1 and a JS `FsmcLcd` class accumulates its register + framebuffer, exactly the path compiled C takes). WS clients attach/detach live; `ext_devices` at create for wired chips (`spi_flash/i2c_eeprom/i2c_oled/lcd/touchscreen/software_spi/`

fsmc_bank/sd_card with peripheral/address/jedec_id/cs/file/data/ width/height/name/size shapes). No TMP102-style pluggable SDD framework string (`-sdd thermometer:...`) — the equivalent here is `register_js_peripheral(base,size,read,write)` (last-wins shadowing) + `-periph-plugin` on the CLI.

11 Part 11: Performance

- Headless **~70M IPS** (200M ~2.7–2.9s, shared-box $\pm 30\%$) with full MPU enforcement; browser ~96 MIPS headless / multi-MIPS interactive. Interpreter-bound (~97% in `rustcpu_run`): no micro-opts without a `cpu-prof` win.
- **No instruction cache, no JIT** (Bramble §12.1–12.3 has both: 64K icache + 16K JIT blocks ~1.5 $\times$, RV icache 99.97%). The fetch path is a raw `read16_raw` + `len(op)` + direct `exec16/exec32` dispatch every instruction — measured fast enough (~70M IPS) that neither structure paid for itself; the only caching is the bus `last-slot` temporal fast path + binary search over sorted slots (§12, `src/bus.rs`).
- **No host threading** (Bramble §12.4: one pthread per core, big-lock + quantum + WFI cond-var + `/tmp/bramble-corepool.reg`). Single-threaded WASM needs no atomics (MPU_ON/DEBUG_HALT/WATCH are plain statics; `Rc/RefCell` throughout with `unsafe Sync`); WFI halts the run loop until the next batch dispatch wakes it.
- Landed wins: per-batch tick 3.8 $\times$ (once per batch, instruction-delta); closed-form timer advance 124 $\times$ (1409 $\rightarrow$ 11 ms); hookless counting ~20%; number (not `BigInt`) counters ~19%; batch 20K free (5 $\times$ latency cut); MPU fast path (30% $\rightarrow$ ~5% via `MPU_ON` mirror + cold outlines + raw fetch); adaptive 20K/50K; `batch_size` override; `PROFILE` env timing (`emu/dma/batch/irq/pin` split + MIPS line in `emulator.js run()`).
- ~1B gate evals/run: ANY per-access call shape costs ~30% in V8 — only zero-call + small-hot-skeleton recovered it (measured 2.6 $\rightarrow$ 3.9 $\rightarrow$ 2.8s across variants).
- Batch-boundary timing: peripherals tick between batches, never mid-batch — tests must be async-style (arm once, poll across batches); only `svc` is synchronous. IRQ latency 20K $\approx$ 1.1 ms.
- Benchmarks: `tests/bench.mjs` (peripheral microbench: bare/GPIO+UART/ NVIC+IRQ/mixed + codeHook overhead), `pkg/bench_dual.mjs` + `pkg/bench_merged.mjs` (legacy Unicorn Path-A A/B: native wins ~3.5 $\times$ `periph39`, ~5.5 $\times$ `blink` — kept for history, non-portable hardcoded Unicorn path inside).

12 Part 12: Design Decisions and Trade-Offs

Decision	What was chosen
Correctness over new peripherals	Every modeled bit proven by a failing-then-passing test; unmodeled
Instruction-budget timing (§5.3)	1 instr = 1 cycle; the one decision everything else follows
Batched DMA + completion-last IRQs	Deterministic, zero-JS, correct order
64-IRQ budget + <code>last_popped</code> fairness	Hot IRQs (TXE) alternate with pendings; bits coalesce intra-batch
Half-reset NRST	Model state only (CPU/RAM reloaded by the driver)
Lenient unknowns	Unknown CAN/SDIO commands succeed benignly; unmapped read
No pin-level SWDIO	Transaction-level DP host API
No JIT/icache	Raw fetch + direct dispatch (~70M IPS is enough)
Single-threaded WASM	Plain statics, <code>Rc/RefCell</code> , no atomics

Decision	What was chosen
No FUSE/FAT/persistence	Images in, images out

13 Part 13: Repository Structure

```

stm32f1-emu/ |--- Cargo.toml / Cargo.lock
Rust crate (cdylib+rlib, wasm-bindgen/js-sys/log/svd-parser) |---
package.json
  npm stm32f1-emu@3.3.0 (exports ., ./emulator, ./cli, ./gdb, ./wasm)
    |--- pkg/
    emulator.js / stm32f1.js / gdbstub.mjs / ws-server.mjs / cli.mjs +
      WASM + .d.ts |
    |--- bench_dual.mjs / bench_merged.mjs      legacy Unicorn Path-A
      benches (history only) |
    '--- index.html              minimal .bin-only demo (no
      presets/events) |--- site/
    Pages root: index.html + worker.js + ws-viewer.html + docs hub +
      docs-src/ mirrors |
    |--- worker.js              module Worker:
    init/run/stop/uart/gpio/USB/I2C/OTG/host-trace protocol |    |---
      board_pins.json
    Arduino aliases (STM32duino 2.12.0: D13/D33/BUT/D17...) |    |---
      sync-docs.mjs
    mirrors 22 docs -> docs-src/ + docs.json (CI drift guard) |    '---
      *.elf / *.bin
    shipped firmware + device images (force-added, build/ ignored) |---
      src/cpu/
    thumb.rs (decoder) / mod.rs (run+exceptions) / mem.rs (FlatMemory) /
      regs.rs |
    + census.rs / isa_tests.rs / core_tests.rs / diffuzz.rs / smoke.rs
      |---
    src/peripherals/              adc afio bkp bootloader can crc dac
      dma dwt exti flash
    fsmc gpio i2c itm |              iwdg mod
      (bus+routing) nvic otg
    pwr rcc rtc scb sdio spi swd sw_spi systick |
              tim
    usart usb wwdg (.rs each) |--- src/ext_devices/
      spi_flash /
    i2c_eeeprom+oled / lcd / touchscreen / fsmc_nor / sd_card / |
    display (FSMC LCD) / usart_probe + mod.rs registry |--- src/bus.rs
    sorted slots + binary search + tick_indices + last-wins overlap |---
      src/system.rs
    WasmSystem (bus+NVIC+DMA queues+events+MPU+SWD) + INSTRUCTION_COUNT
      |---
    src/interrupts.rs              64-IRQ budget intr_next |---
      src/native.rs
    rustcpu_* + swd_* wasm boundary (NATIVE process-global) |---
      src/lib.rs
    all wasm exports (board_*, init, step/process_batch, drain_events,
      injects...) |---
    svd/                          STM32F103.svd + STM32F105xx.svd |---
      tests/

```

```

test_all.mjs (772) + canary + ~50 suites + ~30 firmwares + OTG
bare-metal |
+ census_*.py + fuzz_diff.py + bench.mjs + gdb_live_session.sh
(dev-only) |--- docs/
source docs (this Guide + API + USAGE + PERIPHERALS + ...) |---
template/ / examples/ /
scripts/ |--- docs/STM32F1_Guide.md          this file (website
Guide entry) |---
docs/STM32F1_Technical_Manual.tex pdflatex source (->
STM32F1_Technical_Manual.pdf)
'--- ignore/                                UNTRACKED local reference only
(Bramble docs; never
committed)

```

14 Part 14: Version History

14.1 Version history

Version	Date	Key features
3.3.0	2026-10-05	ADC + TIM wrapper classes, per-batch event ordering, batch introspection
3.2.0	2026-09-26	DMA + clocks/power/debug JS surface, watchdog proofs (772), repo rename
3.1.0	2026-09-12	OTG_FS device/host, Maple DFU, SWD/JTAG slice + GDB Z2/Z3/Z4
3.0.1/3.0.0	2026-09	Real-stack USB (BTABLE-16, PMA-1K, SETUP-ACK, SOF-activity), docs viewer
2.1.0	2026-09	Chips/IDCODE, GDB stub, I2C slave/10-bit, USB SOF/DB/ISO, TIM burst, den
2.0.0	2026-09	Native Rust CPU replaces Unicorn (~3.5×: 200M 9.5→2.7 s)
1.4.0	2026-08	Bus, JS peripherals, GPIO electrical, ADC/FSMC/sleep/SVC, DMA fixes, WS281
1.3.0–0.1.0	2026-08	Init release train (Unicorn era → native cutover; see docs/PATH_B.md)

15 Part 15: External References

15.1 External references (annotated)

Source	Pins down in this manual
ST RM0008 (F103 reference manual)	Every register offset/mask/reset in §6.1–§
ST STM32F103/F105 SVDs (svd/)	Map windows + IRQ numbers (§4.3, §6.1
STM32duino 2.12.0	Arduino core the demos compile against;
CoreMark 1.0	Known-answer CPU check (§7 table); UM
Capstone (test-only)	16/32-bit census oracle (§7 table; tests/
Unicorn 2.1.4 (test-only)	Differential-fuzz oracle (tests/fuzz_dif
wasm-pack 0.14.0 / wasm-bindgen 0.2.126 / Binaryen 132	Byte-exact WASM build (§2.2; CI CODE
xpack-gcc 14.2.1 / arm-none-eabi-gdb 15	Bare-metal OTG demos + live GDB sess
GDB RSP	Command table semantics (§9.1); T05wat
Pico SDK / RP2040+RP2350 datasheets / ARM DDI 0484/0419	Bramble-side references (§7.1 comparison

15.2 Glossary

Term	Meaning
BKPT	Breakpoint instruction (16-bit 0xBE00; GDB Z0 patches it into flash)
BRR	Baud-rate register (USART) / bit-reset register (GPIO) — context disambiguates
CCR	DMA channel configuration register (not TIM capture/compare — also context)
CSR	Control/status register (SysTick CSR, SDIO STA-class, USB CNTR/ISTR family)
DWT	Data Watchpoint and Trace (CYCCNT + comparator slots for GDB Z2/Z3/Z4)
EWI	Early-wakeup interrupt (WWDG counter crossing 0x40)
HSI/HSE/LSI/LSE	High/low-speed internal/external oscillators (8 MHz HSI/HSE modeled; LSI/LSE clock)
IFCR/ISR	DMA interrupt-flag clear / interrupt-status registers
MPU	Memory Protection Unit (8 regions; §5)
PMA	USB packet memory area (1024 B window 0x40006000–0x40006400)
RSP	GDB Remote Serial Protocol (§9.1)
SVD	System View Description XML (map source; <code>svd/</code>)
W1C	Write-1-clears (flag-clear convention: IFCR, ISTR, PR, SR)

15.3 Document history

Date	Change
2026-09-26	First full-manual cut: 15 parts from the repo tree + Bramble shape; <code>scripts/guide_to_tex.py</code>
2026-09-27	Bramble-depth pass: CPU structs + full ISA table + exception/delivery/lookup (§5.1–§5.4), per
2026-10-05	3.3.0 API pass: ADC + TIM wrapper classes (§8.5), per-batch event ordering, <code>getBatchSize/g</code>